

Student Research Paper

CSCI 716 - Software Design
Fall Semester 2025

Addressing Security in Software Design

Submit To:

Pratap Kotala, Ph.D.

Assistant Professor of Practice
Department of Computer Science
Software Engineering
North Dakota State University

Submitted By:

Uchechukwu Johnfisher Uchem

North Dakota State University, USA, Fall 2025

Abstract

In the evolving landscape of software development, security has, certainly, become a principal concern that has received several attentions. In fact, it has become a foundational requirement as modern systems increasingly operate in interconnected and incredibly confrontational environments. In line with this development, this paper, therefore, examines principles, methodologies, and strategies for embedding security early in the software development life cycle, rather than treating it as an addendum, and also explores other solutions to this security problems.

However, key considerations include threat modeling, secure architecture patterns, least-privilege design, input validation, and the integration of automated security tools. Emphasis is placed on designing for resilience, ensuring confidentiality, integrity, and availability through proactive risk assessment and in-depth defense approaches.

When a security-driven decision-making is incorporated from the onset, organizations can reduce vulnerabilities, minimize long-term costs, and build robust systems capable of withstanding evolving cyber security threats.

1. Introduction

The increasing frequency of cyber threats has necessitated a more proactive approach to software design's security. Traditional methods often address security during post-development, leading to vulnerabilities. This paper, therefore discusses the importance of incorporating security into the software design phase.

As software designers and architects develop applications that operate in distributed, cloud-centric, and data-intensive environments, traditional approaches that address security only during implementation or post-deployment are no longer sufficient. Instead, security must be treated as a superficial design objective that is integrated systematically throughout the software development lifecycle.

Emphasis on security-awareness during software design seeks to identify potential vulnerabilities early, incorporate protective architectural patterns, and ensure that security requirements are aligned with functional and performance goals. Principles such as least privilege, defense in depth, secure defaults, and threat-driven design, guide developers in building systems that are resilient to both accidental failures and intentional attacks. At the same time, design methodologies ranging from threat modeling and attack-surface analysis tend to secure architecture frameworks, thereby, providing structured techniques for evaluating risks and making informed design decisions.

Despite the increasing availability of software development tools, guidelines, and best practices for achieving efficient security software design, still security remains very challenging. Factors like rapid development cycles, complex dependency ecosystems, and evolving regulatory requirements create pressures that can lead to overlooked risks or inconsistent practices. Consequently, integrating security into early design phases is essential not only for preventing vulnerabilities but also for reducing long-term maintenance costs and ensuring user trust.

This paper there examines the previous works done with respect to this development, and the present core principles, methodologies, and challenges associated with security in software design, and provides more insight to reveal if the new results or findings are better than the previous works and findings.

By analyzing both conceptual foundations and practical approaches, we could highlight strategies that would enable developers and architects to embed security proactively, rather than reactively, within modern software systems.

2. Background

It is very obvious that security in software design has emerged as a critical area of study as systems have grown in complexity and exposure. Historically, security considerations were often relegated to late stages of development where it was treated as supplemental features rather than integral part of software design goals. This separation between functional design and security engineering led to systems that were functionally robust yet susceptible to a wide range of vulnerabilities. Over time, industry failures, widespread data breaches, and

increasingly severe attacks have facilitated the need for a more holistic, design-centric approach to software security.

However, foundational work in secure computing introduced principles that continue to inform contemporary practice. Saltzer and Schroeder’s seminal design principles; such as least privilege, fail-safe defaults, open design, and psychological acceptability, provide baseline guidance for constructing secure systems. These principles emphasize that security must be embedded into the architecture itself rather than bolted on as an afterthought. Later frameworks, including secure development lifecycle (SDL) models and standardized controls, were built upon this foundation by offering procedural structures for incorporating security from requirements gathering through deployment.

Threat modeling has become a cornerstone of security-aware design. Approaches such as STRIDE, attack trees, and misuse cases help designers anticipate adversarial behavior by enumerating potential threats and analyzing their impact on system architecture. These methods support early detection of design-level weaknesses, enabling mitigation strategies to be evaluated alongside core system requirements. Parallel advances in formal methods, static analysis, and automated verification tools further strengthen the ability to reason about security properties during early-stage design.

Interestingly, modern software ecosystems introduce new security challenges that amplify the need for rigorous design practices. Distributed systems, microservices, mobile platforms, and cloud-native architectures expand the attack surface and introduce new dependency relationships. Additionally, reliance on third-party libraries, open-source components, and continuous integration pipelines also creates supply-chain risks that must be addressed during design time. Consequently, contemporary secure design increasingly requires the integration of architectural patterns for isolation, authentication, authorization, secure communication, and even resilience.

Collectively, these historical developments and methodological frameworks form part of the foundation of security in software design. Understanding this background is indeed, essential for evaluating current best practices and identifying opportunities to advance secure software engineering design that will suitably respond to these evolving technological and threat challenges.

3. Previous Works

3.1 *Secure Design Principles*

Interestingly, early foundational work focused on defining principles that guide secure architectural and design decisions. ***Saltzer & Schroeder (1975)*** introduced classic security principles including the least privilege, fail-safe defaults, economy of mechanism, separation of privilege, complete mediation and many more others. These formed the basis for many design-level security guidelines. These principles were later built upon in security-oriented software engineering textbooks and frameworks, emphasizing integrating security early rather than at the end of the process.

3.2 *Secure Software Development Lifecycles (SSDLCs)*

During the progressive transition process of early software development, researchers and industry groups expanded security from principles into full development methodologies. Microsoft SDL (Howard & Lipner, mid-2000s) emphasized threat modeling, secure coding, and security reviews embedded within each development phase.

OWASP Software Assurance Maturity Model (SAMM) and BSIMM studied real organizations and provided structured maturity models for integrating security activities into design and architecture. According to OWASP, their goal is to be the prime maturity model for software assurance that provides an effective and measurable way for all types of organizations to analyze and improve their software security posture (***OWASP SAMM, 2025***).

Secure Software Development Life Cycle - SSDLC models like CLASP and Secure Unified Process provided prescriptive activities such as architecture risk analysis, abuse-case modeling, and design-level vulnerability identification.

These efforts collectively shifted security to embedding risk analysis and defensive design earlier in software projects.

3.3 *Threat Modeling and Risk Analysis*

A major sub-area of previous work focuses on systematically anticipating and evaluating security threats at the design stage. STRIDE is a threat modeling framework used in computer security to identify potential security threats. Shostack and Microsoft that used this model formalized threat categories as Spoofing, Tampering, Repudiation, and many others, and applies them at architectural diagram level.

DREAD, which is a risk assessment model used in software engineering to evaluate and prioritize potential security threats, CVSS, which means Common Vulnerability Scoring System, is a standardized framework used to assess and rate the severity of software vulnerabilities, and other scoring models supported quantitative or semi-quantitative assessment.

However, academic research also introduced other methods like attack trees, attack graphs, misuse and abuse cases, and attack surface analysis. Later extensions explored automation, model-driven threat modeling, and integrating threat modeling into agile workflows.

Threat modeling remains one of the most influential design-level security techniques.

3.4 *Security Patterns and Reference Architectures*

Having borrowed some knowledge from software design patterns, researchers proposed reusable solutions for recurring security problem. Security design patterns for example authentication, secure logger and role-based access control were cataloged by works such as *Schumacher et al.'s "Security Patterns"*.

Architectural patterns for layered defenses, secure microservices, zero-trust architectures, and distributed systems security appeared in both academic and industry literature.

Pattern-based development frameworks aimed to reduce design flaws by forcing architects to use vetted security solutions.

We can clearly see how this line of work addressed the challenge of reusing expert knowledge during high-level design.

3.5 *Secure Architecture Evaluation Techniques*

Researchers also developed systematic methods to evaluate the security of system architectures which includes Architecture Risk Analysis like the McGraw's work on software security, Architectural tactics for availability, confidentiality, integrity, and resilience, and Architectural trade-off analysis methods (ATAM extensions for security)

All these methods help identify design-level vulnerabilities and trade-offs early on.

3.6 *Standards, Guidelines, and Regulatory Frameworks*

Various industry and government bodies contributed frameworks influencing secure design. Some of these include ISO/IEC 27034 for application security, NIST SP 800-160 for systems security engineering, Common Criteria design-level security requirements and protection profiles, and OWASP Top 10 and CWE lists, which inform design strategies for addressing common weaknesses.

These standards formalize design-level expectations and help organizations achieve compliance and repeatability.

3.7 *Taxonomy of Security-Related Requirements*

This work identifies and defines four distinct types of security-related requirements, and organizes them into a hierarchical taxonomy. It goes further to clarify the often-unclear concept of "security requirements" and emphasizes that security must be considered at the requirements-engineering phase; not just at implementation or testing.

However, this taxonomy aligns security requirements with other “quality” requirements, thereby facilitating their integration in early design and architectural decisions.

3.8 *Taxonomy of Security Considerations and Software Quality.*” ***Communications of the ACM, June 2003.***

This paper argues that security should be treated as a first-class concern in software quality. Security risks like credential theft, denial of service, data exposure, and unauthorized administration threaten operational quality attributes such as reliability, integrity, availability, and correctness.

This work, therefore, provides a systematic framework for mapping security risks onto software quality factors. A typical example is the McCall’s quality model. It also outlines how protection mechanisms like standards, tools, libraries, and administrative controls can mitigate them.

The authors emphasize that security must be integrated from the start of the design process, rather than added ad-hoc after development.

3.9 *Security Patterns and Design Patterns*

Security Patterns-related work by A Systematic Mapping Study - ***Javan Jafari, A., & Rasoolzadegan, A. (2018)***

This mapping study reviewed literature from 1997 to 2017, identifying 403 relevant papers and selecting 274 for detailed analysis. The survey explores classification schemes, pattern notations, pattern evaluation techniques, and usage environments.

The study illustrates the breadth of security pattern research and highlights the challenges in standardizing how patterns are defined, documented, evaluated, and applied in practice.

The authors concluded that while security patterns are widely proposed, their adoption and effective use remain complicated by inconsistent definitions, lack of evaluation, and insufficient integration into software development processes.

3.10 *Secure Design Patterns*

Secure Design Patterns-related work by ***Dougherty, C., Sayre, K., Seacord, R., Svoboda, D., & Togashi, K. (CMU/SEI-2009-TR-010, 2009)***

This report adapts the concept of “design patterns” (initially popular for object-oriented design) to security problems. It defines secure design patterns at multiple levels of abstraction (architecture, design, implementation), aiming to avoid accidental vulnerabilities or mitigate their impact.

The patterns were derived by generalizing existing best practices and extending non-security design patterns to include security-specific functionality. This makes security reusable and applicable across different contexts.

The authors argue that embedding security into design, rather than bolting it on later, reduces the cost and risk associated with vulnerabilities.

3.11 *Survey on Security Patterns*

A Survey on Security Patterns related work by ***Yoshioka, N., Washizaki, H., & Maruyama, K. (2008)***

This earlier survey analyzed existing security-pattern approaches and pointed out that although many patterns have been proposed, integrating them into each phase of software development remains challenging.

It highlights barriers such as context-dependence, difficulty in adapting patterns across different application domains, and the lack of widespread adoption.

3.12 *Systematic Literature Review of Security Pattern*

Systematic Literature Review of Security Pattern Research work by ***Washizaki, H. et al. (2021)***

This review evaluated about 240 peer-reviewed papers and constructed a taxonomy of security pattern research. It reports that since the late 1990s roughly 500 security patterns have been proposed. The authors emphasize that while technical aspects of patterns are well-covered, the overall picture remains fragmented including the different definitions, overlapping categorizations, inconsistent terminology, and limited evidence of real-world adoption.

The study calls for standardization for terminology, evaluation, documentation, and better communication between researchers and practitioners to increase the practical impact of security patterns.

3.13 *Abstract security patterns and the design of secure systems*

Abstract security patterns and the design of secure systems work by **Fernandez, E. B., Yoshioka, N., Washizaki, H., et al. (2022)**

This work argues in favor of identifying security constraints at early stages during requirements analysis, focusing on the semantic aspects of applications rather than implementation or architecture. By defining “abstract security patterns (ASPs)”, the authors aim to link attacker goals, which is a threat, with conceptual defenses before design decisions.

Once ASPs are defined in analysis, they can be refined in design and eventually mapped to software artifacts, thereby, bridging the gap between high-level security requirements and concrete design and implementation.

4 **Recent Work Done and New Findings; Automation, Microservices, CI/CD, and Modern Contexts**

4.1 *Securing Microservices and Microservice Architectures*

As microservice architectures (MSA) grow in popularity, the paper maps existing literature on threats and security mechanisms specific to MSA. Out of 1067 screened papers, 46 were selected for detailed study.

The authors created a lightweight ontology of security patterns for microservices: cataloging threats, mitigation strategies, applicability layers (communication, deployment, infrastructure), and validation techniques.

Their mapping reveals that most research focuses on access control and auditing, while prevention and mitigation like runtime defense, fault tolerance, and other deeper layers like communication and deployment remain underexplored.

4.2 *Authentication and authorization in microservice-based systems*

This paper surveys major architecture patterns for implementing authentication and authorization in microservice-based systems. It discusses how different patterns apply depending on environment characteristics, and evaluates their trade-offs which are in the form of advantages and disadvantages.

Such pattern-level guidance helps architects choose appropriate designs early to reduce attack surface and manage identity/permission issues in distributed, modular systems.

4.3 *Enhancing Security in Software Design Patterns and Antipatterns*

Recognizing limitations of traditional static-analysis, this work proposes a novel framework that uses Large Language Models (LLMs) to detect security vulnerabilities related to design patterns and antipatterns. By combining security heuristics like “least privilege” and “input validation” with LLM-based reasoning about code structure and behavior, the framework aims to catch subtler design-level vulnerabilities. Typical examples include insecure communication, improper data handling, or unauthorized state changes, especially in codebases evolving under CI/CD.

This work suggests a new direction in software security design which may include leveraging AI and ML not just for runtime monitoring or anomaly detection, but for design-time vulnerability detection and prevention.

4.4 *Intent-Driven Secure System Design*

This paper addresses the complexity of designing secure networked systems. Manual design and audit become increasingly difficult as systems grow. Consequently, the authors propose an automated design methodology that includes security requirements, aimed to reduce human error and resource demand in architecture design.

This work reflects a shift from manual “pattern-by-pattern” design to more automated, tool-assisted security design, which could help scale security for large or fast-changing systems.

4.5 *Security-by-Design Adoption Remains Superficial in Practice*

Despite widespread promotion of security-by-design principles, recent assessments show that many development teams apply them only at a policy level. Deeper integration such as continuous threat modeling and security-driven architecture refactoring remains limited due to time constraints, insufficient security expertise, and lack of organizational incentives.

4.6 *Shift-Left Security Reduces Vulnerabilities but Requires Cultural Alignment*

New empirical studies indicate that shifting security tasks earlier in the development cycle significantly reduces post-release vulnerabilities. However, effectiveness depends heavily on team culture. Organizations with stronger collaboration between developers and security engineers see a greater reduction in severe vulnerabilities compared to those relying solely on automated tools.

4.7 *AI-Assisted Code Generation Creates New Attack Surfaces*

The rise of AI-assisted coding tools introduces novel risks. Findings show that AI-generated code may inherit insecure patterns from training data. These artifacts are often subtle and evade traditional static analysis tools, highlighting the need for new detection techniques tailored specifically to AI-generated code.

4.8 *Threat Modeling Must Evolve to Address Cloud-Native Complexities*

Traditional threat modeling techniques, such as STRIDE, do not fully capture the dynamic nature of microservices, container orchestration, and ephemeral infrastructure. New model extensions are being proposed to incorporate runtime risks, API attack surfaces, and supply-chain dependencies, demonstrating significantly more accurate risk predictions for cloud-native systems.

4.9 *Secure Defaults Outperform Security Training Alone*

There is glaring evidence that embedding secure defaults in frameworks and tooling results in a larger reduction in vulnerabilities than developer training alone. Examples include automatic input sanitization, secure authentication templates, and opinionated security configurations. This finding underscores the importance of designing security into the ecosystem, not just into developer behavior.

4.10 *Zero-Trust Architecture Improves Software Resilience Beyond Network Boundaries*

Applying zero-trust principles which proposes "never trust, always verify," at the software design level, not just the network level, has shown promising results. Incorporating continuous authorization, granular identity boundaries, and micro-segmentation into software architecture significantly reduces lateral movement opportunities for attackers.

4.11 *Real-Time Observability Enhances Security Responsiveness*

Modern telemetry and observability tools have revealed that embedding real-time monitoring hooks during the design phase enables earlier attack detection and faster mitigation. This approach is more effective than traditional logging added late in development, which often fails to capture contextual data needed for forensic analysis.

4.12 *Supply-Chain Security Is Now a Dominant Risk Factor*

Studies reveal that the majority of critical vulnerabilities in modern software originate from third-party dependencies rather than proprietary code. New approaches, such as automated dependency health scoring, continuous SBOM (Software Bill of Materials) validation, and cryptographic signing, show significant promise in reducing supply-chain-related risks.

5.0 **Synthesis: Previous Works Vs Current Works and Improvements.**

5.1 Security should be considered from the very beginning. foundational works (*Firesmith, Taxonomy-of-Security, "Software Security Is Software Reliability"*) emphasize that security is not an afterthought, instead, it must be built into requirements, architecture, and design, to affect quality attributes and design decisions from the start.

5.2 Security patterns is a major vehicle for reusable security knowledge. Since the late 1990s, researchers have proposed many "security design patterns" to capture recurring security challenges and solutions. Pattern catalogs like Secure Design Patterns, and core pattern books could help disseminate and reuse secure design principles across contexts.

5.3 There are still many challenges in the adoption, standardization, and evaluation security patterns in software design. Systematic reviews like mapping studies highlight that despite hundreds of proposed patterns, their real-world adoption is low, definitions are inconsistent, evaluation is limited, and there is no universal standard for documenting or applying them.

5.4 As more threat landscape and software architectures are evolving, so new security designs are expected to evolve. With microservices, distributed architectures, CI/CD pipelines, and cyber systems all at risk, traditional security-by-design approaches are being re-examined and extended. New methods including automated design, LLM-based detection, and threat-aware architectural patterns are also emerging.

5.5 With the increase for holistic, lifecycle-awareness for security design, recent works pushes for embedding security across the entire lifecycle, from requirements, through design and implementation, to deployment and maintenance, especially in CPS or industrial systems.

6.0 Recommendations

If your work aims to design secure software architectures, the pattern-based and taxonomy-based literature gives you reusable building blocks and a terminology to describe threats and defenses systematically.

If you're focusing on early-phase security requirements and analysis, the work on security requirements taxonomy and abstract security patterns (ASPs) shows how to embed security as this tends to be more effective than retroactive fixes.

If your domain is modern like microservices, CI-CD, cyber-physical, distributed, recent surveys show both useful architectural patterns and significant gaps provide an opportunity for novel contributions, especially in evaluation, standardization, and real-world adoption.

As software complexity grows, there's a trend toward automation tool-assisted design like AI and LLM-supported detection that are very useful if you aim for scalable, maintainable security solutions rather than manual design ad-hoc.

7.0 Conclusion

In conclusion, previous work on addressing security in software design includes foundational principles, complete development processes, threat modeling techniques, security patterns, formal verification methods, and global standards. Collectively, this body of research emphasizes “security by design”. Building systems where security concerns are integrated into architecture and design from the very beginning, rather than applied retroactively is the way forward in order to achieve secure software design.

However, addressing security in software design is a fundamental responsibility that ensures the integrity, confidentiality, reliability, trust, and long-term sustainability of software systems. When security considerations are integrated early in the design process, organizations can significantly reduce vulnerabilities, lower remediation costs, and protect both users and critical data. Effective security requires a blend of proactive threat modeling, adherence to secure design principles, continuous testing, and a commitment to dealing with emerging threats. Ultimately, building security into the foundation of software fosters resilient systems that can withstand modern cyber risks and deliver safe, dependable experiences for all users.

Finally, several works have been previously done, and more are still in progress presently as presented in this research paper to tackle security issues in Software Design.

References

- [1] Firesmith, D. (2005). A Taxonomy of Security-Related Requirements. SEI White Paper.
- [2] Dougherty, C., Sayre, K., Seacord, R., Svoboda, D., & Togashi, K. (2009). Secure Design Patterns. CMU/SEI-2009-TR-010.
- [3] Yoshioka, N., Washizaki, H., Maruyama, K. (2008). A Survey on Security Patterns. Prog. in Informatics, no. 5.
- [4] Washizaki, H. et al. (2021). Systematic Literature Review of Security Pattern Research. Information 12, 1.
- [5] Javan Jafari, A., Rasoolzadegan, A. (2018). Security Patterns: A Systematic Mapping Study.
- [6] Fernandez, E. B., Yoshioka, N., Washizaki, H., et al. (2022). Abstract Security Patterns and the Design of Secure Systems. Cybersecurity 5, 7.
- [7] Hannousse, A., Yahiouche, S. (2020). Securing Microservices and Microservice Architectures: A Systematic Mapping Study.
- [8] Barabanov, A., Makrushin, D. (2020). Authentication and authorization in microservice-based systems: survey of architecture patterns.
- [9] Andrade, R., Torres, J., & Ortiz-Garcés, I. (2025). Enhancing Security in Software Design Patterns and Antipatterns: A [10] Framework for LLM-Based Detection. Electronics 14 (3), 586.
- [10] D. Basin, J. Doser, and T. Lodderstedt. 2006. Model Driven Security. ACM TISSEC.
- [11] M. Howard and S. Lipner. 2006. The Security Development Lifecycle. Microsoft Press.
- [12] D. Jackson. 2006. Software Abstractions: Logic, Language, and Analysis. MIT Press.
- [13] G. McGraw. 2006. Software Security: Building Security In. Addison-Wesley.
- [14] R. Epstein and G. McGraw. 2002. The Secure Unified Process.
- [15] OWASP. 2020. Software Assurance Maturity Model (SAMM).
- [16] B. Schneier. 1999. Attack Trees. Dr. Dobb's Journal.
- [17] M. Schumacher et al. 2005. Security Patterns: Integrating Security and Systems Engineering. Wiley.
- [18] A. Shostack. 2014. Threat Modeling: Designing for Security. Wiley.